Copula Risk Modelling Project

R

August 10, 2026

Contents

Contents

1 Introduction

In this project, I use R to calculate the Value at Risk (VaR) and the Average Value at Risk (AVaR) at the 99% CI for a portfolio of 4 stocks on 22-02-2023. The VaR_α is defined as the potential loss level such that only α% of parametric losses would be greater. The AVaR_α is generally the mean of the losses that would exceed the VaR_α. This involves specifically the linear model and a Gaussian copula with *t*-distributed marginals. For convenience, we assume an unchanging portfolio of investments into the 4 risky assets. The period of data we have consists of 1,258 observations between 22-02-2018 and 21-02-2023.

Based on Figure 1, we can see that the bulk of our portfolio was invested into Assets 2 and 3, with 1 and 4 averaging just below £4,000 for the duration of the trading period. This makes sense looking at Figure 1, where Assets 2 and 3 have the highest closing prices, while 1 and 4 have similar prices and lower volatility over the period.

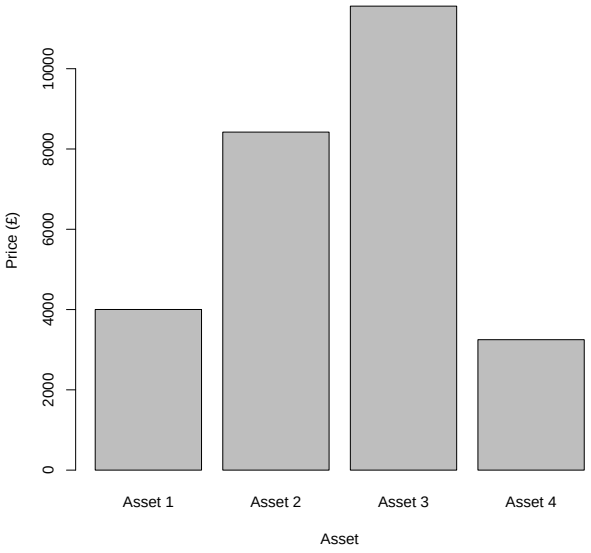

Figure 1: (Mean) Stock Values in Portfolio over duration of trading

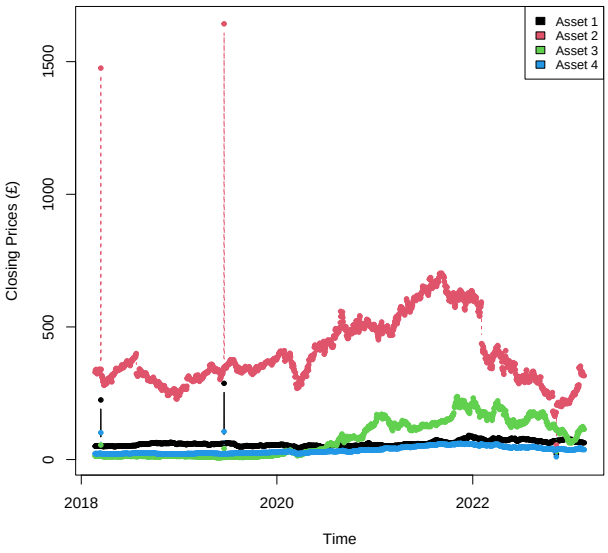

Figure 2: Close Prices for Stock Portfolio

2 Cleaning the data

After loading up the data, we want to convert it to logged returns for more reliable parameter estimation. Observing Figure 3, we can see that the distributions are generally distributed regularly around 0, but that all of the assets experience anomalous spikes on 6 dates, where the absolute logged stock returns all exceed 1. To this end, we can clean the data of anomalies by filtering out all returns exceeding 1. If we observe the filtered dataset in Figure 4, we can see that the two periods that generally had the most variance were in early 2020, around the first COVID-19 lockdown in the UK, and around mid-2022, perhaps as the post-lockdown ”bounce-back”.

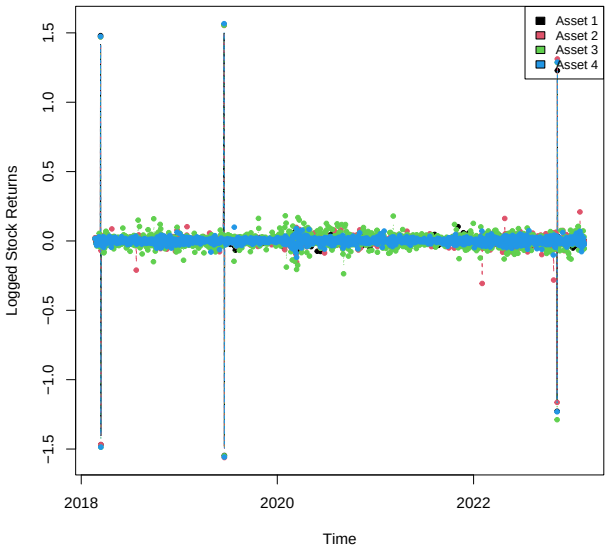

Figure 3: Logged Returns for Assets

3 Variance-Covariance linear model

We can apply the variance-covariance method to estimating VaR and AVaR. We can sample the 2 years prior to our sampling day, giving us 501 days worth of data. The variance-covariance method assumes that stock returns are normal, and approximates the evolutionary process using a linear model. For short term periods like a single day however, this should be reasonably accurate. The model works by assuming the existence of a pricing function f, such that the profit V can be calculated as a deterministic function of random variables, called risk factors.

$$V_t = f(t, \mathbf{Z}_t)$$

The return then becomes:

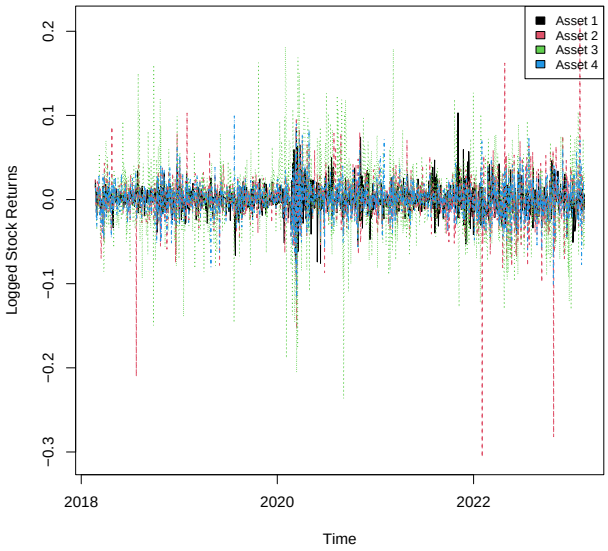

Figure 4: Cleaned Logged Returns for Assets

$$\triangle V_t^{\text{lin}} := \frac{\delta f}{\delta t}(r, \mathbf{Z}_t)\triangle t + \sum_{i=1}^{d} \frac{\delta f}{\delta z_i}(t, \mathbf{Z}_t)\triangle Z_t^i$$

We can practically obtain this for a one day VaR by obtaining a mean and covariance matrix estimate, and calculating the loss for the $(1-\alpha)\%$ confidence interval using the normal distribution. The AVaR can be calculated as the density of the distribution.

Observing block 1, we can see that the estimated VaR is £2003.50, and the AVaR is £2293.39. These two values are very close together, reflective of one of the biggest problems with any Gaussian model of stock returns, as extreme outcomes become very rare parametrically despite our heuristic knowledge that poor economic situations are far more volatile than stable markets (closer to the mean).

Table 1: Linear model VaR and AVaR estimates
The Value at Risk estimated from the linear model is £2003.50, and the Average Value at Risk estimated from the model is £2293.39.

4 Copula fitting

If we want to model distributions with heavier tails, we can apply the *t*-distribution. This section is based on (McNeil et al. 2015), with methods reconstructed based on Chapter 7. Tidyverse was used for data manipulation methods and MASS was used for multivariate normal distribution sampling. We can model our ‘marginal‘ stock

distributions, each individual asset, according to their own *t*-distributions. We can then fit a Gaussian 'copula', an effective joint cumulative distribution function, in order to obtain more accurate and deeper tails.

Our first objective is fitting the marginals, for which we need to choose the degrees of freedom. This comes out to 4 (Appendix 10) for all 4 assets. Armed with fitted *t*-distributions, we can convert our returns to uniform distributions by applying the cumulative distribution function for the respective asset. This gives us the distributions in Figure 5, which are recognisably uniform, implying that the models are well fitted.

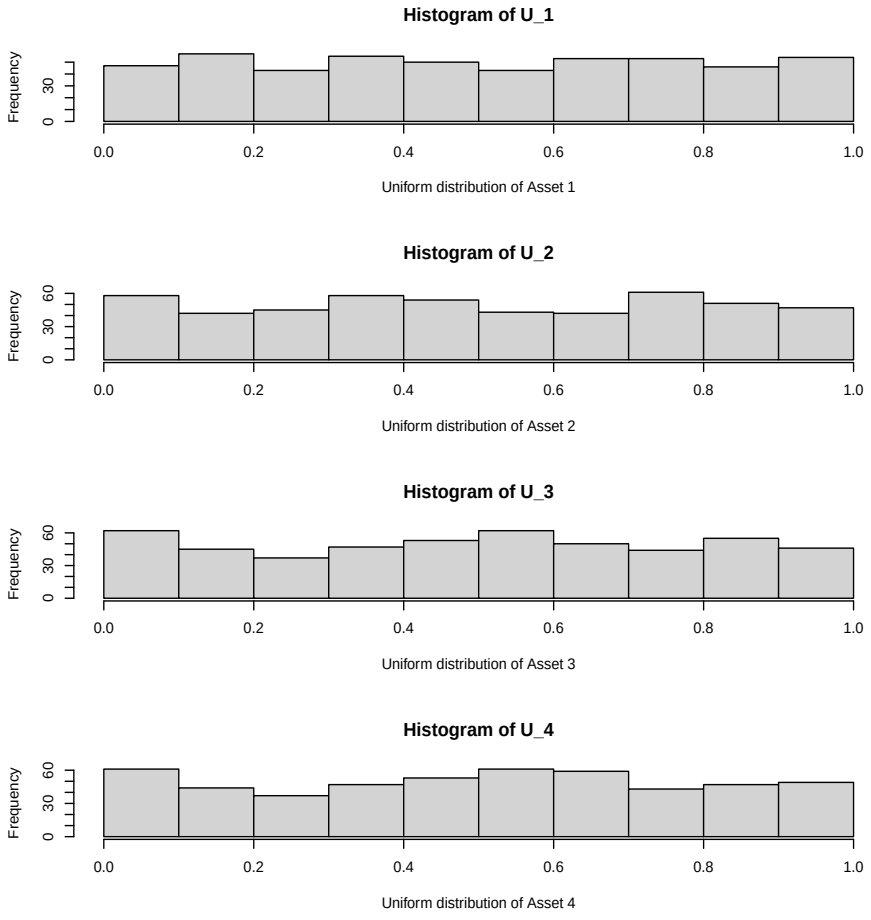

Figure 5: Uniform distributions for original Assets

We can now estimate the correlation matrix directly, as the weighted crossproduct of the uniforms (the covariance), divided by the variances of each asset. We assume the distributions centre on 0, so the means are simply a 1×4 vector of 0's. Observing block 4, we can see that the estimated correlation matrix was successfully constructed, with the highest correlation between stocks 4 and 2, and the lowest between 1 and 3.

```
[1] "Our estimated correlation matrix: "
           [,1]      [,2]       [,3]      [,4]
[1,] 1.00000000 0.1596394 0.01487549 0.1671441
[2,] 0.15963939 1.0000000 0.43511877 0.7302483
[3,] 0.01487549 0.4351188 1.00000000 0.4861356
[4,] 0.16714412 0.7302483 0.48613555 1.0000000
Our copula means: [1] 0 0 0 0
```

The copula is fitted, and the marginals are fitted too. We can now freely generate values from the joint distribution and convert them back into the original distributions. We can observe the estimated stock returns in Figure 6, where the distributions look accurate to the sample distribution. Further, we can se the increased joint marginal distributions in Figure 7. For stocks with low correlation, like Assets 1 and 3, we can see the distributions look seemingly randomly distributed according to a mean 0 Gaussian distribution. However, for more highly correlated stocks like Assets 2 and 4, the joint distribution has far heavier tails, reflecting a higher instance of extreme negative outcomes.

We can calculate the portfolio value for any of these simulated possibilities, and chart them out as a loss distribution by deducting the day before from the simulated portfolio value. Observing Figure 8, we can see that the losses are vaguely normally distributed, but do have heavy tails, indicating a distribution that better reflects the probability of extreme outlier cases. We can see that the final values for VaR and AVaR also reflect this, where the copula estimated VaR is £2,158.79 and the AVaR is £2,837.97, the linear model VaR is only £2,003.50 and the AVaR is £2,293.39. The major disparity between the AVaR especially is closely related to the Gaussian distribution of stock returns grossly underestimating the frequency of outliers.

```
[1] "The copula estimated VaR is £-2158.79, and the copula"
[1] "estimated AVaR is £-2837.97 at the 0.01 confidence"
[1] "interval."

[1] "The linear model estimated VaR is £2003.50, and"
[1] "the linear \n model estimated AVaR is £2293.39"
[1] "at the 0.01 confidence interval."
```

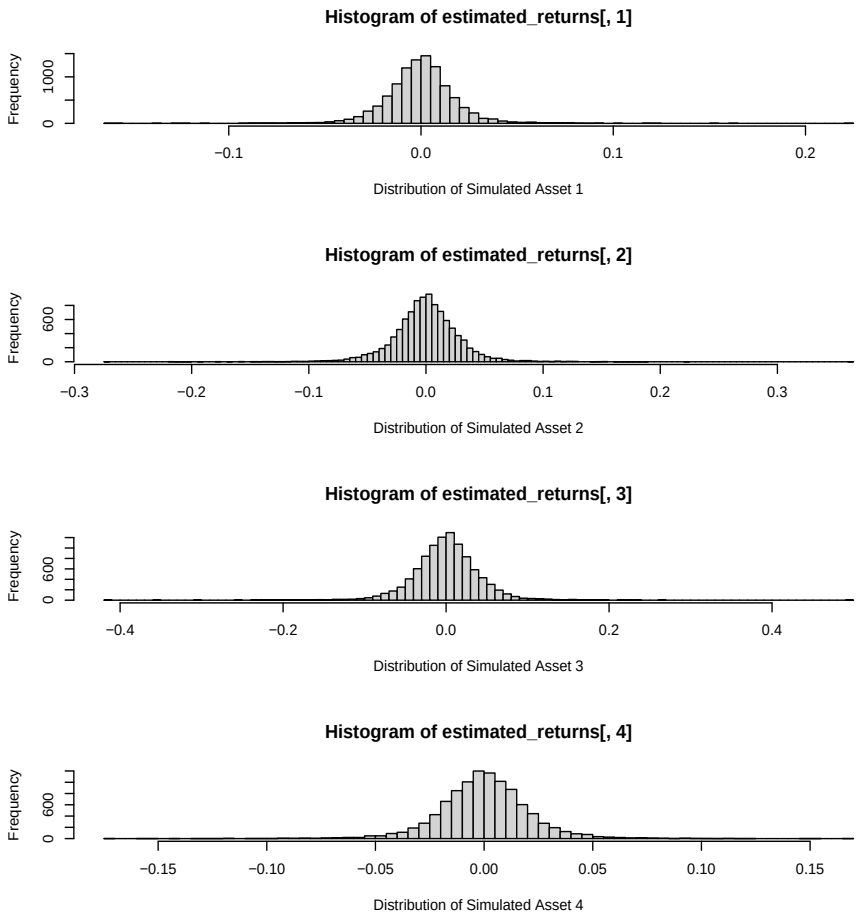

Figure 6: Simulated Stock Return Distributions

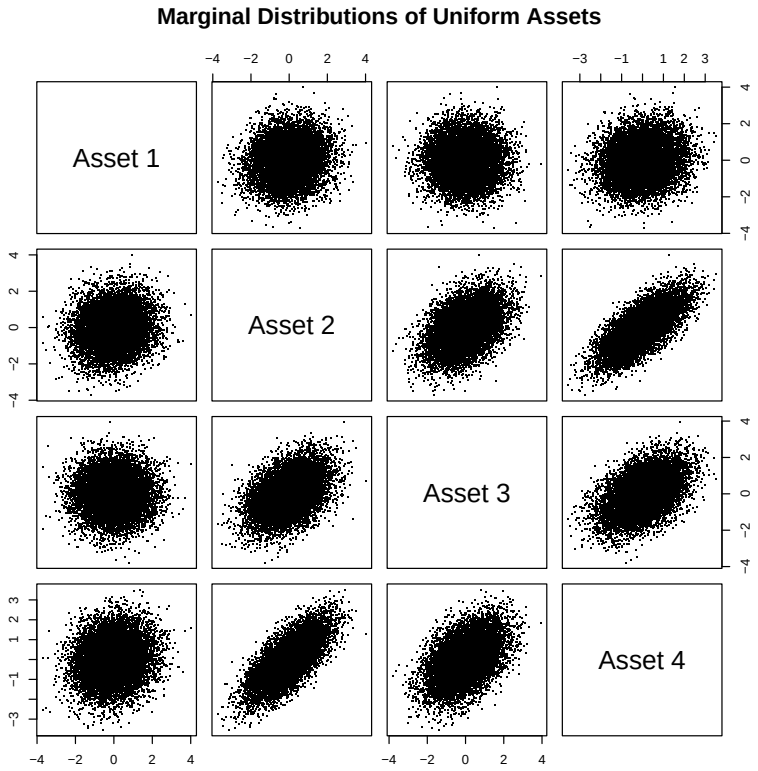

Figure 7: Joint Marginal Distributions for Simulated Assets

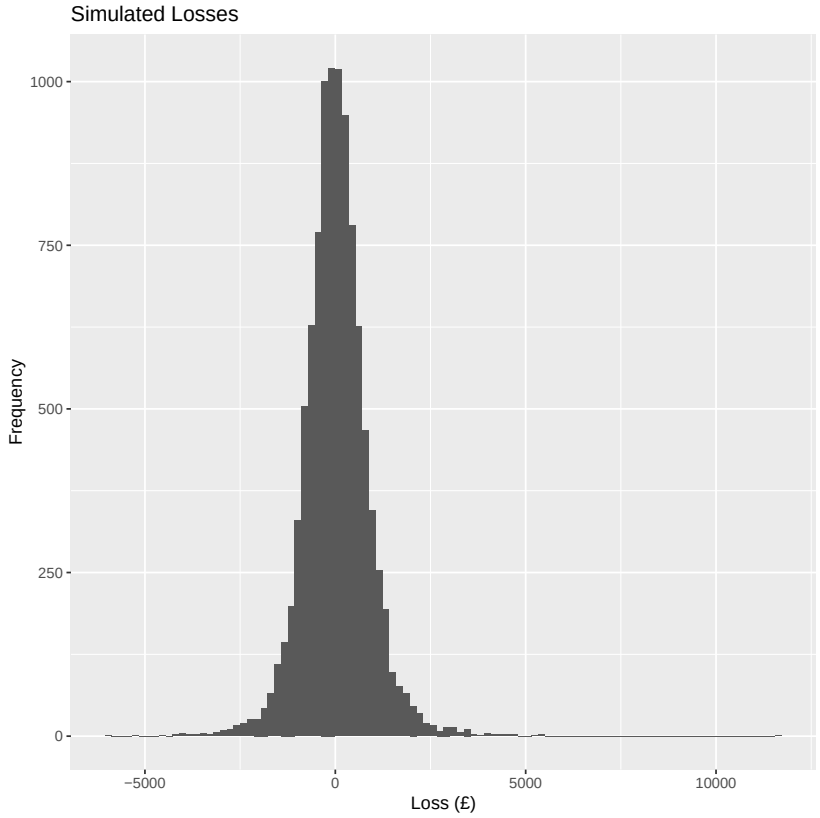

Figure 8: Copula Simulated Loss Distribution

5 Stress testing

We can now consider a real set of stocks and evaluate their risk under possible tail scenarios. We have the stocks NVIDIA (NVDA), Apple (AAPL), USD Dollar Fund Unhedged, Evolve Enchanced Yield Bond (on the XTSE), and Pimco ETF (on the XNYS), collected from the Bing stock ticker in Excel. We also have the market return, risk-free rate, and tech market return rates over the period (2024/01/02-2026/05/18), taken from Fama and French market data. There were four missing values in the Evolve Enchanged Yield bond, which were interpolated using the zoo library.

We can again clean the data and run our linear approximation and copula risk modelling, with the results presented in Table 2.

Table 2: Portfolio VaR and expected shortfall prior to articifial stress

Method	$\text{VaR}_{0.99}$	$\text{AVaR}_{0.99}$
Variance covariance	£2918.42	£3358.89
Gaussian Copula (*t*-marginals)	£3101.86	£4427.88

We can stress test our stocks using the Capital Asset Pricing Model (CAPM):

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_{i,m}(r_{m,t} - r_{f,t}) + \epsilon_{i,t}, \tag{1}$$

where R_i is the value of the stock at time i, $R_{f,i}$ is the risk-free rate at time i, α_i is the return above (or below) market excess returns, and β_i measures the relationship between how the stock varies with the market at large, and ϵ is some disturbance term. In our case, both of our stocks are tech stocks, so we can also include a third term including the tech industry excess return. We can easily implement this as a linear regression, with an augmented Dickey-Fuller test and post-regression residual normality testing confirming that the linear regression model accuracy assumptions are satisfied.

$$r_{i,t} - r_{f,t} = \alpha_i + \beta_{i,m}(r_{m,t} - r_{f,t}) + \beta_{i,I}I_t + \epsilon_{i,t} \tag{2}$$

where I_t is the industry excess return and $\beta_{i,I}$ is the corresponding beta. With these fitted to each stock, we can also fit the CAPM for tech excess industry returns based on equation 1, allowing us to simulate a reduced tech market in the case where tech returns fall *x%*, the market return falls *y%*, and the risk-free rate falls *z%*:

$$I_{i,t} - r_{f,t} = \alpha_i - x + \beta_{i,m}(r_{m,t} - r_{f,t} - y) - z + \hat{\epsilon}_{i,t},$$

where $\hat{\epsilon}_{i,t}$ is the observed residuals in the market data, and $\tilde{I}_{i,t} = I_{i,t} - r_{f,t}$ is our new reduced tech industry excess return. We can then substitute these into 2, as:

$$r_{i,t} - r_{f,t} = \alpha_i - x + \beta_{i,m}(r_{m,t} - r_{f,t} - y) + \beta_{i,I}\tilde{I}_t - z + \hat{\epsilon}_{i,t}.$$

These equations define a method for simulating different negative market conditions for stocks, such as falls in interest rates, the market excess return, or tech downfalls.

In this project, I considered three potential situations:

1. tech downturn where the market return drops 8%, the risk free rate drops 2%, and the tech market drops 10%,

2. heavy tech downturn where market return drops 10%, the risk free rate drops 5%, and the tech market drops 20%,
3. market downturn where market return drops 25%, risk free rate drops 5%, and tech market drops 5%.

Using a Gaussian Copula with *t*-marginals, the new values at risk were:

Scenario	$VaR_{0.99}$	$AVaR_{0.99}$
1	£5172.16	£6408.59
2	£8264.95	£9498.29
3	£8196.56	£9459.90

Compared to Table 2, we can observe that the potential losses are far higher, almost rising up to a tenth of the original value of the portfolio in the case of $AVaR_{0.99}$ in Scenario 3. It is interesting to observe how similar the impact of decreases in the market return and tech industry excess returns impact are, indicating that a key direction to expand this kind of testing would be to identify better how the two could compound each other. It could also be worth studying particular downturns like the DotCom crisis to test real life stress situations that particularly target the tech markets.

6 Appendix

Code blocks evaluated in org-babel (version: `release_9.7.37`) on Mac Tahoe 26.3.1, using R version 4.5.1. Only CRAN library dependencies are `tidyverse` and `here`, although `here` is optional and only used for file pathing.

```
set.seed(1234) # document seeding
suppressMessages(library(tidyverse))
suppressMessages(library (here)) # here just enables reliable
    pathfinding
suppressMessages(library(MASS))  # MASS provides multivariate
    Gaussian simulations
```

Listing 1: Library Calls for Program

```
close_prices <- read_csv(here("data", "Data_Student_201605543.
    csv"))

shares <- read_csv(here("data", "share_numbers.csv"))

my_shares <- shares %>% filter(`Student ID` == 201605543)
colnames(my_shares) <- c("SID", "Asset 1", "Asset 2", "Asset 3"
    , "Asset 4")
```

```
share_vals <- as.numeric(colMeans(close_prices[-1])) * as.
    numeric(my_shares[-1])

barplot(share_vals, names.arg = colnames(my_shares[-1]), xlab =
     "Asset", ylab = "Price (£)")
```

Listing 2: Visualisation of Stock Value in portfolio

```
matplot(x = close_prices$Date, y = close_prices[-1], type="b",
    pch=20, xlab="Time", ylab="Closing Prices (£)")
legend("topright", colnames(close_prices[-1]), col = seq_len(
    ncol(close_prices[-1])),cex=0.8,fill=seq_len(ncol(close_
    prices[-1])))
```

Listing 3: Close prices visualisation

```
pclose <- log(close_prices[,-1])
risk_factors = lapply(pclose, diff)
dates <- close_prices$Date[1:(length(close_prices$Date)-1)]
risk_factors <- tibble( # I just like tibbles
    `Date`    = dates,
    `Asset_1` = risk_factors$"Asset 1",
    `Asset_2` = risk_factors$"Asset 2",
    `Asset_3` = risk_factors$"Asset 3",
    `Asset_4` = risk_factors$"Asset 4"
)
dim(risk_factors)
```

Listing 4: Conversion to logged returns

```
matplot(x = risk_factors$Date, y = risk_factors[-1], type="b",
    pch=20, xlab="Time", ylab="Logged Stock Returns")
legend("topright", colnames(close_prices[-1]), col = seq_len(
    ncol(close_prices[-1])),cex=0.8,fill=seq_len(ncol(close_
    prices[-1])))
```

Listing 5: Stock Return Distributions

```
risk_factors <- risk_factors %>% dplyr::filter(Asset_1 < 1) %>%
    dplyr::filter(Asset_1 > -1) # these are all the same
        outliers

dim(risk_factors)
```

Listing 6: Removing Anomalies from dataset

```
matplot(x = risk_factors$Date, y = risk_factors[-1], type="l",
    pch=20, xlab="Time", ylab="Logged Stock Returns")
legend("topright", colnames(close_prices[-1]), col = seq_len(
    ncol(close_prices[-1])),cex=0.8,fill=seq_len(ncol(close_
    prices[-1])))
```

Listing 7: Cleaned Stock Return Distributions

```
start_date_1 <- dmy("21-02-2021")
end_date_1 <- dmy("21-02-2023")

load_dates_1 <- risk_factors %>% dplyr::filter(Date > start_
    date_1) %>%
  dplyr::filter(Date < end_date_1)
dim(load_dates_1)
```

Listing 8: Date Range Isolation

```
## sample values
load_means <- colMeans(load_dates_1[sapply (load_dates_1, is.
    numeric)]) # our means vector

load_covs <- cov(load_dates_1[-1]) # our covariance matrix

final_date <- as.data.frame(load_dates_1)[dim(load_dates_1)[1],
     1]
final_close <- close_prices %>% dplyr::filter(Date == final_
    date)
## close prices for the day before our predicted day

## parameter estimation
alpha = 0.01

lin_mu <- as.numeric(my_shares[,-1] * final_close[,-1]) %*% as.
    vector(load_means)

lin_sigma <- sqrt((as.numeric(my_shares[,-1] * final_close
    [,-1])) %*%
                  as.matrix(load_covs) %*% (as.numeric(my_
                      shares[,-1] * final_close[,-1])))

## calculating VaR and AVaR
lin_VaR <- -lin_mu + lin_sigma * qnorm (1-alpha)
lin_AVaR <- -lin_mu + lin_sigma * dnorm(qnorm (1-alpha))/alpha
sprintf("The Value at Risk estimated from the linear model is £
    %.2f, and \n the Average Value at Risk estimated from the
    model is £%.2f.", lin_VaR, lin_AVaR)
```

Listing 9: Linear Approximation for VaR and AVaR

```
fitting_marginals <- function (series) {
    ## just a set of test dfs
    trial_dfs <- c(2, 4, 10, 16, 32, 64)
    df_log_lik <- c(1:6)

    for (i in c(1:6)) {
        ## MASS t distribution fitting
        fit <- fitdistr(
            series,
            "t",
            df = trial_dfs[i], # optional df
            start = list(
                m  = median(series),
                s  = IQR(series) / 2
            ),
            lower = c(m = -Inf, s = 1e-8)
        )

        df_log_lik[i] = fit$loglik
    }

    return (trial_dfs[which.max(df_log_lik)])
}
```

```
## all of these came out as 4 df being the best for me
fitting_marginals(load_dates_1$Asset_1)
fitting_marginals(load_dates_1$Asset_2)
fitting_marginals(load_dates_1$Asset_3)
fitting_marginals(load_dates_1$Asset_4)

fit_1 <- fitdistr (load_dates_1$Asset_1, "t", df = 4)
fit_1$estimate

fit_2 <- fitdistr (load_dates_1$Asset_2, "t", df = 4,
                   start = list(
                       m  = median(load_dates_1$Asset_2),
                       s  = IQR(load_dates_1$Asset_2) / 2),
                   lower = c(m = -Inf, s = 1e-8))
fit_2$estimate

fit_3 <- fitdistr (load_dates_1$Asset_3, "t", df = 4)
fit_3$estimate

fit_4 <- fitdistr (load_dates_1$Asset_4, "t", df = 4,
                   start = list(
                       m  = median(load_dates_1$Asset_4),
                       s  = IQR(load_dates_1$Asset_4) / 2),
                   lower = c(m = -Inf, s = 1e-8))
fit_4$estimate
```

Listing 10: Fitting marginal t-distributions

```
U_1 <- pt((load_dates_1$Asset_1 - fit_1$estimate[1])
          / fit_1$estimate[2], df = 4)
U_2 <- pt((load_dates_1$Asset_2 - fit_2$estimate[1])
          / fit_2$estimate[2], df = 4)
U_3 <- pt((load_dates_1$Asset_3 - fit_3$estimate[1])
          / fit_3$estimate[2], df = 4)
U_4 <- pt((load_dates_1$Asset_4 - fit_4$estimate[1])
          / fit_4$estimate[2], df = 4)

U <- cbind(U_1, U_2, U_3, U_4)

par(mfrow = c(4,1))
hist(U_1, xlab = "Uniform distribution of Asset 1")
hist(U_2, xlab = "Uniform distribution of Asset 2")
hist(U_3, xlab = "Uniform distribution of Asset 3")
hist(U_4, xlab = "Uniform distribution of Asset 4")
par(mfrow = c(1,1))
```

Listing 11: Original Asset Uniform Distributions

```
Y <- qnorm(U) # conversion to normal distribution

sigma_hat <- crossprod(Y) / nrow(Y) # covar calculation

Det <- diag(1 / sqrt(diag(sigma_hat))) #var calculation
P_hat <- Det %*% sigma_hat %*% Det     # corr calculation
```

```
print("Our estimated correlation matrix: ")
print(P_hat)

c_means <- rep(0,4)
cat("Our copula means: ")
print(c_means)
```

Listing 12: Estimating Copula Parameters

```
simulations <- 10000
alpha <- 0.99
day_value <- close_prices %>% dplyr::filter (Date == end_date_
    1)

## take a joint estimate
joint_estimate <- MASS::mvrnorm(n = simulations, mu = c_means,
    Sigma = P_hat)

## convert to marginals by applying inverse transform
marginals_1 <- qt(pnorm(joint_estimate[,1]), df = 4) * fit_1$
    estimate[2] + fit_1$estimate[1]
marginals_2 <- qt(pnorm(joint_estimate[,2]), df = 4) * fit_2$
    estimate[2] + fit_2$estimate[1]
marginals_3 <- qt(pnorm(joint_estimate[,3]), df = 4) * fit_3$
    estimate[2] + fit_3$estimate[1]
marginals_4 <- qt(pnorm(joint_estimate[,4]), df = 4) * fit_4$
    estimate[2] + fit_4$estimate[1]

estimated_returns <- cbind(marginals_1, marginals_2, marginals_
    3, marginals_4)

par(mfrow = c(4, 1))
hist(estimated_returns[,1], breaks = 100, xlab = "Distribution
    of Simulated Asset 1")
hist(estimated_returns[,2], breaks = 100, xlab = "Distribution
    of Simulated Asset 2")
hist(estimated_returns[,3], breaks = 100, xlab = "Distribution
    of Simulated Asset 3")
hist(estimated_returns[,4], breaks = 100, xlab = "Distribution
    of Simulated Asset 4")
par(mfrow = c(1, 1))
```

Listing 13: Sampling from the Copula

```
colnames(joint_estimate) <- c("Asset 1", "Asset 2", "Asset 3",
    "Asset 4")

pairs(joint_estimate, pch = ".", main = "Marginal Distributions
     of Uniform Assets")
```

Listing 14: Cross Plots for Joint Distributions in Uniform Asset simulation

```
original_Lstocks <- day_value[-1] %>% as.numeric %>% log #
    logged values of final closing price before date
```

```
## adds logged final price to estimated returns
changed_vals <- t(apply (estimated_returns, 1, function(x) x +
    original_Lstocks))

estimated_stocks <- exp(changed_vals)

estimated_value <- estimated_stocks %*% as.numeric(my_shares
    [-1])
current_value <- as.numeric(day_value[-1]) %*% as.numeric(my_
    shares[-1])

colnames(estimated_stocks) <- c("Asset 1", "Asset 2", "Asset 3"
    , "Asset 4")

estimated_losses <- apply (estimated_value, 1, function(x) x -
    current_value)
##colname(estimated_losses) <- ("Portfolio Loss")

## unfortunately could not get base histogram to render this
tibble_losses <- as_tibble(estimated_losses)
colnames(tibble_losses) <- ("Simulated Losses (£)")

ggplot(as_tibble(estimated_losses), aes(value)) + geom_
    histogram(bins = 100) + ggtitle("Simulated Losses") + xlab("
    Loss (£)") + ylab("Frequency")
```

Listing 15: Simulated Loss Distribution

```
sorted_ELosses <- sort(estimated_losses)
copula_VaR <- quantile (sorted_ELosses, probs = 1 - alpha, type
     = 7)
copula_AVaR <- mean(estimated_losses[estimated_losses <= copula
    _VaR])

sprintf("The copula estimated VaR is £%.2f, and the copula",
        copula_VaR)
sprintf("estimated AVaR is £%.2f at the %.2f confidence",
        copula_AVaR, 1-alpha)
print("interval.")
cat("\n")
##cat("\n")

sprintf("The linear model estimated VaR is £%.2f, and",
        lin_VaR)
sprintf("the linear \n model estimated AVaR is £%.2f",
        lin_AVaR)
sprintf("at the %.2f confidence interval.",
        1-alpha)
```

Listing 16: Final VaR and AVaR estimates for both models

7 References

McNeil, Alexander J., Rüdiger Frey, and Paul Embrechts. 2015. *Quantitative Risk Management: Concepts, Techniques and Tools*. Princeton Series in Finance. Princeton University Press. https://www.perlego.com/book/738023/quantitative-risk-management-concepts-techniques-and-tools-revised-edition-pdf.